Kosmocrates HYPHAE Master Specification

v0.4

Persistent Morphogenic Corpus Cartography and Norm-Genome Assimilation for the Kosmocrates Workbench

Author: Sebastian Klemm

Architecture consolidation draft prepared for implementation planning

Document status: v0.4 recursive design draft

Date: 2026-05-29

HYPHAE v0.4 extends HYPHAE v0.3 from a run-local CubeSwarm assimilation subsystem into a persistent, tri-temporal, certified, morphogenic corpus cartography layer. It preserves the v0.3 non-copying and evidence-first doctrine while adding CorpusCartography, StructuralCrystals, AssimilationCertificates, NormGenes, HostTargetCollapsePlans and morphogenic governance.

Contents

Chapter 1

Executive Summary

HYPHAE v0.4 is not a replacement for HYPHAE v0.3. It is the persistent morphogenic extension of the v0.3 CubeSwarm pipeline.

HYPHAE v0.3 already defines a host-bound, evidence-first and non-copying assimilation system:

```
HostCube
-> SourceCubes
-> CubeSwarm
-> CubeMandorla
-> CompositeSupportCube
-> HostTargetDelta
-> StructuralYield
-> GateCascade
-> CandidateEvidenceRecord
```

HYPHAE v0.4 turns that run-local path into cumulative structural learning:

```
v0.3 output
-> StructuralCrystalCandidate
-> DualFabricGateCascade
-> AssimilationCertificate
-> CorpusCartographyUpdate
-> NormGeneCandidate
-> HostTargetCollapsePlan
-> MorphogenicCorpusUpdate
```

The central thesis is:

> HYPHAE v0.4 converts run-local structural assimilation into persistent, replayable, certified corpus-scale structural learning.

It remembers what worked, what failed, what matured, what decayed, what should be superseded, what can be certified, what can become a NormGene, and in which order a host repository should collapse its remaining structural voids.

HYPHAE v0.4 remains:

- non-copying,
- evidence-first,
- host-bound,

- replay-governed,
- Foundry-bound,
- Kosmocrates-governed,
- and explicitly non-authoritative by existence.

A CorpusCartography hit may influence retrieval, ranking and planning. It must not bypass gates. A NormGene may guide future runs. It is not a trusted norm. A StructuralCrystal may persist as certified structure. It is not executable code. A CollapsePlan may guide Workbench implementation planning. It is not mutation authority.

Chapter 2

Normative Language and Definitions

The keywords must, must not, should, may are to be interpreted as normative requirement levels.

Primary Terms

HostCube The host repository projected into a layered 5D RepositoryCube with embedded CodeHDAG, mesh, semantic fibers, temporal fibers and TopologicalVoidMap.

SourceCube A transient cube projection of an acquired external or approved source.

CubeSwarm A deterministic ensemble of SourceCubes aligned against a HostCube.

CompositeSupportCube The evidence-weighted support envelope created by Monolithic Coagulation of a CubeSwarm.

StructuralYield A typed v0.3 output derived from MotifExtraction.

StructuralCrystal A certified, immutable, replayable structural artifact derived from a StructuralCrystalCandidate and an AssimilationCertificate.

NormGene An evidence-backed, fitness-tracked, replayable structural invariant candidate. A NormGene is not a trusted norm.

CorpusCartography The persistent, evidence-bound structural map over SourceCubes, motifs, StructuralCrystals, NormGenes, negative evidence, conflicts and relations.

HostTargetCollapsePlan An ordered plan for collapsing HostVoids and HostTargetDeltas into Workbench-ready structural implementation steps.

MorphogenicCorpusUpdate A governed update to active CorpusCartography views that preserves committed history.

Non-Authority Terms

- CorpusCartography may remember. It does not authorize.
- NormGenes may guide. They do not govern.
- StructuralCrystals may certify structure. They do not execute.
- CollapsePlans may plan. They do not mutate.
- Certificates may permit target surfaces. They do not bypass Foundry or Kosmocrates governance.

Chapter 3

Source-System Neutralization and Invariant Extraction

HYPHAE v0.4 is derived from several existing architecture documents, but none of those systems is imported as-is. Each source system is neutralized into reusable invariants.

NEUT-001
A source architecture may contribute invariants, operators, schemas, failure models or metrics. It must not be imported as its original domain system.

NEUT-002
Domain metaphors are not normative unless translated into typed HYPHAE objects.

NEUT-003
Every imported invariant must be mapped to a HYPHAE v0.4 responsibility.

Source System	Original Surface	Extracted Invariant	HYPHAE v0.4 Object
MCCE	Crypto cartography / signals	Persistent self-expanding topological learning graph; topology as model; retention tiers	CorpusCartography, CorpusRelation, StructuralCrystalArchive
ISLS	Semantic ledger substrate	Append-only replay, Semantic Crystal, Dual-Fabric certificate, non-retroactivity	StructuralCrystal, Assimilation-Certificate, ReplayProof
Tri-Temporal Helix	Projection engine and phase carriers	Null-center, intrinsic/extrinsic time, phase ladder, carrier migration, Mandorla overlap	TriTemporalRunClock, PhaseLadder, CarrierMigration

Hypercube Decomposer	Dimensional collapse code generator	Freiheitsgrade, singularities, bisection, synchronized groups, validation after collapse	HostTargetCollapsePlan, CollapseGraph
Infogenetik	Norm genetics and corpus condensation	Resonite, topology as compressed artifact, Norm as invariant, fitness trace	Resonite, NormGene, NormFitnessTrace
ISLS v3 Norm System	Cross-layer norm generator	Norm layers, activation, compatibility, Atom/Molecule/Organism/Ecosystem levels	NormAssembly, NormLayerCoverage
Norm Composition	Systems-of-systems generator	Bridge norms, configurations, Verbund, landscape composition	BridgeMotif, HostCluster, LandscapeTargetDelta

This neutralization keeps v0.4 focused: the artifacts enrich HYPHAE's foundation, not the other way around.

Chapter 4

Core Doctrine v0.4

DOCTRINE-040
HYPHAE v0.4 remembers structure, not authority.

DOCTRINE-041
CorpusCartography may influence retrieval, ranking, scoring, planning and maturity. It must not bypass GateCascade, DualFabric certification, Foundry, Kosmocrates governance, license, taint or provenance.

DOCTRINE-042
A NormGene is not a trusted norm.

DOCTRINE-043
A StructuralCrystal is a certified structural artifact, not executable code.

DOCTRINE-044
Morphogenic updates may alter active views, but must not mutate committed truth.

DOCTRINE-045
Consensus, source count, CubeSwarm resonance, CorpusCartography maturity, NormGene fitness or StructuralCrystal support must not override authority, license, taint, safety, governance or Foundry requirements.

Scope

HYPHAE v0.4 provides:

- persistent CorpusCartography;
- tri-temporal runtime and phase migration;
- StructuralCrystal certification;
- NormGene maturation;
- HostTargetCollapsePlan;
- morphogenic corpus governance;

- optional NormAssembly and BridgeMotif composition.

HYPHAE v0.4 does not provide:

- raw external code import through the standard path;
- autonomous host mutation;
- direct trusted memory promotion;
- direct governance mutation;
- direct tool authorization;
- Foundry simulation.

Chapter 5

v0.3 Compatibility Layer

HYPHAE v0.4 preserves the v0.3 run-local assimilation automaton.

```
RunDescriptor
-> HostBinding
-> HostCube
-> TopologicalVoidMap
-> DeficiencyVector
-> SourceIntent
-> SourceFrontierGraph
-> AcquisitionEligibility
-> SourceAcquisitionCapability
-> SourceEvidence
-> CodeObservation
-> CodeHDAG
-> SourceCubeWorker
-> SourceCube
-> CubeSwarm
-> CubeMandorla
-> MonolithicCoagulation
-> CompositeSupportCube
-> HostTargetDelta
-> MotifCandidate
-> StructuralYield
-> GateCascade
-> AssimilationDecision
-> IntegrationOutput
-> Metrics / LearningTrace
```

v0.4 extension points are explicit:

v0.3 Point	v0.4 Extension
After DeficiencyVector	CartographyPrecheck retrieves prior motifs, NormGenes, negative evidence and known conflicts.
Around CubeSwarm	TriTemporalRunClock, PhaseLadder and CarrierMigration govern phase evolution.
After StructuralYield	StructuralCrystalCandidate, DualFabricGateCascade and AssimilationCertificate certify durable value.

After HostTargetDelta	HostTargetCollapsePlan orders structural improvement.
After IntegrationOutput	CorpusCartographyUpdate, NormGeneCandidate and MorphogenicCorpusUpdate persist learning.

COMPAT-040
v0.4 must not create a second independent assimilation pipeline. It extends the v0.3 pipeline through declared extension points.

Chapter 6

HYPHAE v0.4 Run Automaton

The canonical v0.4 run is:

```
R0  Load RunDescriptor
R1  Initialize TriTemporalRunClock
R2  Bind HostCube
R3  Compute VoidMap + DeficiencyVector
R4  CartographyPrecheck
R5  Reuse prior Motifs / NormGenes if sufficient
R6  Build SourceFrontier if needed
R7  Acquire / parse / lower / SourceCube
R8  CubeSwarm + Mandorla + Coagulation
R9  MotifExtraction -> StructuralYield
R10 v0.3 GateCascade
R11 Build StructuralCrystalCandidates where durable value exists
R12 DualFabricGateCascade
R13 AssimilationCertificate
R14 Update CorpusCartography
R15 Birth/update NormGenes
R16 Build HostTargetCollapsePlan
R17 Route Workbench / Foundry / Ledger outputs
R18 MorphogenicCorpusUpdate
R19 RunReport + Metrics
```

Operational Principle

The v0.4 run may skip external acquisition if CartographyPrecheck returns sufficient prior certified structure. It may reduce SourceFrontier scope if negative evidence indicates repeated dead ends. It may migrate carriers if friction or shock makes the active path poor. It may emit a CollapsePlan even when no code changes are performed.

RUN-040
Every v0.4 run should initialize a TriTemporalRunClock and produce a clock digest.

RUN-041
Every durable v0.4 output must be reachable from RunDescriptor, HostBinding, EvidenceBundle, GateTrace and, where applicable, AssimilationCertificate.

Chapter 7

CorpusCartography

CorpusCartography is the persistent structural map introduced by v0.4.

CARTO-000
CorpusCartography is not trusted memory. It is an append-only, evidence-bound, replayable structural map over observed sources, SourceCubes, CorpusRelations, Motifs, NormGenes, StructuralCrystals, conflicts and negative evidence.

```
pub struct CorpusCartography {
    pub id: Digest,
    pub scope: CorpusScope,

    pub source_index: SourceCubeIndex,
    pub relation_index: CorpusRelationIndex,
    pub motif_index: MotifIndex,
    pub norm_gene_index: NormGeneIndex,
    pub structural_crystal_archive: StructuralCrystalArchive,
    pub negative_evidence_index: NegativeEvidenceIndex,
    pub conflict_index: ConflictIndex,

    pub storage_tiers: CorpusStorageTiers,
    pub replay_root: Digest,
    pub policy_profile_id: Digest,
    pub created_at: Timestamp,
    pub updated_at: Timestamp,
}
```

Scopes

```
pub enum CorpusScope {
    LocalHostProject(Digest),
    WorkspaceFamily(Digest),
    OrganizationPrivateCorpus(Digest),
    ApprovedMirrorCorpus(Digest),
    PublicOpenSourceCorpus,
    DomainProfile(String),
}
```

Scope separation prevents private corpora, public mirrors and local projects from collapsing into a single ambiguous authority surface.

Layer Model

L0 Corpus Discovery Layer Repositories, mirrors, registries, metadata and probes.

L1 Corpus Relation Layer Similarity, conflict, fork, same ecosystem, shared motif and risk relations.

L2 Corpus HDAG / Cartography Layer Persistent structure map over SourceCubes, motifs, HostVoid matches and recurring topologies.

L3 Crystal Emission Layer StructuralCrystalCandidates, NormGeneCandidates and AntiPatternCrystals.

L4 Archive / Substrate Layer Append-only cold archive, replay manifests, digest verification and retention.

`CARTO-AUTH-001`
CorpusCartography must not be treated as trusted memory.

`CARTO-AUTH-002`
A CorpusCartography hit may reduce search cost or increase candidate priority, but it must not replace GateCascade.

Chapter 8

CorpusEntity, CorpusRelation and Lifecycle

```
pub struct CorpusEntity {
    pub id: Digest,
    pub entity_kind: CorpusEntityKind,
    pub scope: CorpusScope,
    pub canonical_digest: Digest,

    pub first_seen_run: Digest,
    pub last_seen_run: Digest,

    pub evidence_refs: Vec<EvidenceRef>,
    pub taint: TaintLabel,
    pub license_status: LicenseUseStatus,

    pub lifecycle_state: CorpusEntityLifecycleState,
}
```

```
pub enum CorpusEntityKind {
    SourceRepository,
    SourceSnapshot,
    SourceCube,
    CodeHDAG,
    MotifCandidate,
    StructuralYield,
    NormGeneCandidate,
    StructuralCrystalCandidate,
    AntiPatternEvidence,
    NegativeEvidence,
    ConflictRegion,
    BridgeMotifCandidate,
}
```

Lifecycle States

```
pub enum CorpusEntityLifecycleState {
    Observed,
    Indexed,
    Active,
```

```
    Mature,
    Dormant,
    Reactivated,
    Superseded,
    Rejected,
    Quarantined,
    Archived,
}
```

CorpusRelation

```
pub struct CorpusRelation {
    pub id: Digest,
    pub from: CorpusEntityRef,
    pub to: CorpusEntityRef,

    pub relation_kind: CorpusRelationKind,
    pub strength: Q16,
    pub confidence: Q16,
    pub maturity: RelationMaturity,

    pub first_seen_run: Digest,
    pub last_updated_run: Digest,

    pub evidence_refs: Vec<EvidenceRef>,
    pub negative_evidence_refs: Vec<NegativeEvidenceRef>,
    pub decay: RelationDecayState,
}
```

```
pub enum CorpusRelationKind {
    SimilarTopology,
    SharesMotif,
    SharesNormGene,
    SharesFailureClass,
    SharesDependencyPattern,
    SharesTestStructure,
    SharesInterfaceBoundary,
    SharesSecurityBoundary,
    SourceDerivedFromFork,
    SameEcosystem,
    SameFramework,
    SameDomainRole,
    ConflictsWith,
    Supersedes,
    Refines,
    Duplicates,
    LowYieldNeighbor,
    LicenseRiskNeighbor,
    InjectionRiskNeighbor,
    BridgeCandidate,
    CrossHostContractCandidate,
}
```

CARTO-REL-001
Every CorpusRelation must have relation kind, strength, confidence, maturity and

evidence.

CARTO-DECAY-001
CorpusRelations should decay rather than disappear.

Chapter 9

Cartography Indices and Precheck

SourceCubeIndex

```
pub struct SourceCubeIndex {
    pub cubes: BTreeMap<Digest, SourceCubeIndexEntry>,
    pub by_source_digest: BTreeMap<Digest, Vec<Digest>>,
    pub by_profile: BTreeMap<Digest, Vec<Digest>>,
    pub by_motif_kind: BTreeMap<MotifKind, Vec<Digest>>,
    pub by_deficiency_kind: BTreeMap<DeficiencyKind, Vec<Digest>>,
    pub by_license_status: BTreeMap<LicenseUseStatus, Vec<Digest>>,
    pub by_taint: BTreeMap<TaintLabel, Vec<Digest>>,
}
```

MotifIndex

```
pub struct MotifIndexEntry {
    pub motif_id: Digest,
    pub motif_kind: MotifKind,
    pub structural_core_digest: Digest,

    pub source_support: CorpusSupport,
    pub host_fit_history: HostFitHistory,
    pub foundry_history: FoundryOutcomeHistory,

    pub promoted_norm_gene: Option<NormGeneRef>,
    pub rejected_reason: Option<NegativeEvidenceReason>,

    pub evidence_refs: Vec<EvidenceRef>,
}
```

NegativeEvidenceIndex

```
pub enum NegativeEvidenceKind {
    LowYieldSource,
    DuplicateSource,
    LicenseIncompatible,
    LicenseUnknownHighRisk,
    SecretDetected,
```

```
    InjectionRisk,
    ParseIneligible,
    UnsupportedLanguage,
    LowHostAlignment,
    HighConflict,
    FailedFoundryValidation,
    KnownBadMotif,
    QueryDeadEnd,
}
```

CartographyPrecheck

```
pub struct CartographyPrecheck {
    pub id: Digest,
    pub run_id: Digest,
    pub host_cube_id: Digest,
    pub deficiency_vector_id: Digest,

    pub prior_motif_hits: Vec<MotifIndexEntry>,
    pub prior_norm_gene_hits: Vec<NormGeneRef>,
    pub relevant_negative_evidence: Vec<NegativeEvidenceEntry>,
    pub known_conflicts: Vec<ConflictRegionRef>,
    pub recommended_frontier_seeds: Vec<QuerySeed>,
    pub reduced_search_budget: Option<AcquisitionBudget>,
}
```

```
pub enum CartographyPrecheckDecision {
    ExistingEvidenceSufficient,
    SearchReduced,
    SearchRequired,
    SearchRequiresDifferentProfile,
    AbortKnownUnsafe,
    HumanReviewRecommended,
}
```

CARTO-PRE-001
CartographyPrecheck may reduce or redirect SourceFrontier. It must not skip safety, license, taint, provenance, GateCascade or Foundry requirements.

Chapter 10

CorpusCartographyUpdate and ReplayManifest

A v0.4 run ends with a CorpusCartographyUpdate if it produced reusable positive or negative evidence.

```
pub struct CorpusCartographyUpdate {
    pub id: Digest,
    pub run_id: Digest,
    pub cartography_id: Digest,

    pub added_entities: Vec<CorpusEntity>,
    pub updated_relations: Vec<CorpusRelationUpdate>,
    pub new_negative_evidence: Vec<NegativeEvidenceEntry>,
    pub new_crystal_candidates: Vec<StructuralCrystalCandidateRef>,
    pub matured_norm_genes: Vec<NormGeneRef>,

    pub replay_manifest: CartographyReplayManifest,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

```
pub struct CartographyReplayManifest {
    pub id: Digest,
    pub cartography_before_digest: Digest,
    pub update_digest: Digest,
    pub cartography_after_digest: Digest,

    pub run_descriptor_digest: Digest,
    pub operator_versions: BTreeMap<String, String>,
    pub policy_profile_digest: Digest,

    pub input_evidence_bundle_ids: Vec<Digest>,
    pub output_evidence_bundle_ids: Vec<Digest>,

    pub canonicalization_profile_id: Digest,
}
```

CARTO-REPLAY-001
Given the same prior CorpusCartography digest, RunDescriptor, evidence bundles,

operator versions and policy profile, a CorpusCartographyUpdate should produce the same after-digest.

CARTO-REPLAY-002
All replay-relevant operator versions must be pinned.

Chapter 11

TriTemporalRunClock

TriTemporalRunClock provides v0.4 with a clear time model.

```
pub struct TriTemporalRunClock {
    pub id: Digest,
    pub run_id: Digest,

    pub null_center: NullCenter,
    pub intrinsic_time: IntrinsicRunTime,
    pub extrinsic_time: ExtrinsicCommitTime,

    pub phase_ladder: PhaseLadder,
    pub migration_policy: CarrierMigrationPolicy,

    pub clock_digest: Digest,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

NullCenter

```
pub struct NullCenter {
    pub id: Digest,
    pub run_descriptor_digest: Digest,
    pub host_cube_digest: Digest,
    pub policy_profile_digest: Digest,
    pub cartography_before_digest: Option<Digest>,
    pub canonical_seed: Digest,
}
```

NullCenter is the immutable ordering anchor of a HYPHAE run. It accumulates no evolving state.

TIME-NC-001
NullCenter must not store evolving run state.

TIME-NC-002
Deterministic tie-breaks in PhaseLadder, Worker ordering, SourceCube ordering and migration ordering should derive from NullCenter and canonical digests, not wall-clock order.

Intrinsic and Extrinsic Time

IntrinsicRunTime is internal, exploratory and reversible. ExtrinsicCommitTime is durable, replay-relevant and authoritative for CorpusCartography updates.

```
pub enum IntrinsicRunEventKind {
    FrontierExpanded,
    SourceRanked,
    AcquisitionScheduled,
    SourceEvidenceProduced,
    CodeHDAGBuilt,
    SourceCubeProjected,
    ProjectionConflictDetected,
    MandorlaUpdated,
    CoagulationAttempted,
    FrictionDetected,
    ShockDetected,
    CarrierMigrationProposed,
    CarrierMigrationExecuted,
}
```

```
pub enum ExtrinsicCommitKind {
    RunStarted,
    FrontierClosed,
    AcquisitionEvidenceCommitted,
    SourceCubeCommitted,
    CompositeSupportCommitted,
    StructuralYieldCommitted,
    GateCascadeCommitted,
    AssimilationCertificateCommitted,
    CorpusCartographyUpdated,
    RunClosed,
}
```

TIME-EXT-001
Only ExtrinsicCommit events may mutate durable CorpusCartography, Ledger, StructuralCrystalArchive, RetrievalIndex, Workbench ContextPack, Agenda, FoundryQueue or HumanReviewQueue.

Chapter 12

PhaseLadder and CarrierMigration

PhaseLadder allows HYPHAE to maintain prepared alternative carriers.

```
pub struct PhaseLadder {
    pub id: Digest,
    pub active_slot: PhaseSlotId,
    pub slots: Vec<PhaseSlot>,
    pub max_active_slots: usize,
    pub phase_policy: PhasePolicy,
}
```

```
pub enum PhaseSlotKind {
    Primary,
    ReserveFrontier,
    ReserveProfile,
    SafetyFallback,
    HighPrecisionLowRecall,
    LowRiskLocalOnly,
    DocumentationOnly,
    TestsOnly,
    PriorEvidenceOnly,
}
```

CarrierMigration

```
pub enum CarrierMigrationKind {
    FrontierMigration,
    AcquisitionModeMigration,
    CubeProfileMigration,
    MotifProfileMigration,
    CorpusScopeMigration,
    SafetyFallbackMigration,
    HumanReviewMigration,
}
```

```
pub enum CarrierMigrationDecision {
    StayOnActiveCarrier,
    MigrateToReserve,
    SplitLoadAcrossCarriers,
    ReduceToEvidenceOnly,
```

```
    ReduceToPriorEvidenceOnly,
    RequireHumanReview,
    AbortRun,
}
```

Friction and Shock

```
pub struct FrictionReport {
    pub low_yield_rate: Q16,
    pub policy_rejection_rate: Q16,
    pub license_uncertainty_rate: Q16,
    pub parse_failure_rate: Q16,
    pub conflict_growth_rate: Q16,
    pub resonance_decay: Q16,
    pub budget_burn_rate: Q16,
    pub total_friction: Q16,
}
```

```
pub struct ShockReport {
    pub secret_detection_event: bool,
    pub injection_cluster_event: bool,
    pub license_block_event: bool,
    pub source_ecosystem_shift: bool,
    pub acquisition_policy_violation: bool,
    pub repeated_sandbox_failure: bool,
    pub severe_conflict_spike: bool,
    pub total_shock: Q16,
}
```

MIG-001
CarrierMigration may change search, acquisition, profile or motif strategy.

MIG-002
CarrierMigration must not downgrade safety policy.

MIG-003
CarrierMigration must preserve provenance and emit a migration trace.

MIG-004
CarrierMigration must not erase negative evidence from the abandoned carrier.

Chapter 13

Dual Projection and Phase-aware CubeSwarm

v0.4 strengthens v0.3 alignment through bidirectional projection.

```
pub struct DualProjectionCarrier {
    pub id: Digest,

    pub host_to_source_projection: ProjectionTrace,
    pub source_to_host_projection: ProjectionTrace,

    pub overlap: ProjectionOverlap,
    pub mandorla_support: Option<SupportRegionRef>,

    pub coherence: Q16,
    pub conflict: Q16,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

DUAL-PROJ-001
High-confidence SupportRegion should require both host-to-source need fit and source-to-host structural fit.

DUAL-PROJ-002
A motif that is source-strong but host-weak may become corpus evidence, but should not become Workbench-usable HostTargetDelta without HostAlignment.

TriTemporalCubeSwarm

```
pub struct TriTemporalCubeSwarm {
    pub id: Digest,
    pub base_swarm_id: CubeSwarmRef,
    pub clock_id: TriTemporalRunClockRef,

    pub active_phase: PhaseSlotId,
    pub phase_local_swarms: Vec<PhaseLocalSwarm>,
    pub migrations: Vec<CarrierMigrationRef>,
```

```
    pub intrinsic_trace: Vec<IntrinsicRunEventRef>,
    pub extrinsic_commits: Vec<ExtrinsicCommitRef>,
}
```

```
pub struct PhaseLocalSwarm {
    pub phase_slot: PhaseSlotId,
    pub source_cube_ids: Vec<Digest>,
    pub local_mandorla: Option<CubeMandorlaRef>,
    pub local_composite_support: Option<CompositeSupportCubeRef>,
    pub local_conflicts: Vec<ConflictRegionRef>,
    pub local_metrics: PhaseLocalMetrics,
}
```

Phase-local results must not become durable truth without ExtrinsicCommitGate.

Chapter 14

StructuralCrystalCandidate

A StructuralCrystalCandidate is a hardened candidate for long-lived corpus structure.

```
pub struct StructuralCrystalCandidate {
    pub id: Digest,
    pub run_id: Digest,

    pub source_yield_id: Digest,
    pub candidate_kind: StructuralCrystalKind,

    pub condensed_region: CodeHDAGFragment,
    pub motif_core: MotifStructuralCore,
    pub topology_signature: TopologySignature,
    pub cube_signature: CubeSignature,

    pub constraint_program: ConstraintProgram,
    pub stability_score: Q16,
    pub corpus_support: CorpusSupport,

    pub host_alignment: Option<HostAlignment>,
    pub phase_evidence: PhaseEvidenceSummary,
    pub conflict_summary: ConflictSummary,

    pub evidence_chain: Vec<EvidenceRef>,
    pub operator_versions: BTreeMap<String, String>,

    pub certificate_requirement: CertificateRequirement,
}
```

```
pub enum StructuralCrystalKind {
    MotifCrystal,
    AntiPatternCrystal,
    NormGeneCrystal,
    NormAssemblyCrystal,
    BridgeMotifCrystal,
    FailureRepairCrystal,
}
```

Yield vs Candidate vs Crystal

StructuralYield Typed output from MotifExtraction. It may be run-local.

StructuralCrystalCandidate A yield stable enough to be certified for long-term corpus use.

StructuralCrystalRecord A committed, immutable, replayable structural artifact in the archive.

`CRYSTAL-001`
Discovery does not create a StructuralCrystal.

`CRYSTAL-002`
StructuralYield may become StructuralCrystalCandidate only if it has typed motif core, complete evidence chain, topology signature, corpus support, operator versions and certificate requirement.

Chapter 15

ConstraintProgram

A ConstraintProgram is the structural recognition and validation shape of a candidate.

```
pub struct ConstraintProgram {
    pub id: Digest,
    pub operators: Vec<ConstraintOperatorRef>,
    pub canonical_serialization_digest: Digest,
    pub operator_versions: BTreeMap<String, String>,
    pub evaluation_order: Vec<Digest>,
}
```

```
pub enum ConstraintOperatorKind {
    RoleInvariant,
    EdgeInvariant,
    FiberInvariant,
    LayerInvariant,
    PhaseInvariant,
    HostVoidCoverageInvariant,
    ConflictBound,
    LicenseTaintBound,
    FoundryOutcomeBound,
}
```

Example:

```
RouteIntegrationTestCrystal:
- require Role(Route)
- require Role(Test)
- require Edge(Test -> Route, Tests)
- require FixtureFiber(TestFixture -> Test)
- require NegativeCase coverage for invalid payload/auth
- forbid raw source copying
- require Foundry pass before high-trust promotion
```

CONSTRAINT-001
ConstraintProgram must be canonical, versioned and deterministic.

CONSTRAINT-002
ConstraintProgram must not contain raw external source code.

Chapter 16

DualFabricGateCascade

DualFabricGateCascade separates structural validity from operational admissibility.

```
pub struct DualFabricGateCascade {
    pub id: Digest,
    pub run_id: Digest,
    pub crystal_candidate_id: Digest,

    pub structural_fabric: GateFabricCertificate,
    pub operational_fabric: GateFabricCertificate,

    pub seam_checks: Vec<SeamCheck>,
    pub mirror_consistency: MirrorConsistencyReport,
    pub holistic_decision: DualFabricDecision,

    pub evidence_refs: Vec<EvidenceRef>,
}
```

Structural Fabric

Structural Fabric checks whether the artifact is valid as structure:

- evidence chain complete;
- CodeHDAG fragment valid;
- MotifStructuralCore typed;
- CubeSignature valid;
- ConstraintProgram canonical;
- CorpusSupport sufficient;
- conflict below threshold;
- replay digest stable;
- operator versions pinned.

DF-STRUCT-001
Structural Fabric must fail if motif core is untyped, CodeHDAG fragment is invalid, ConstraintProgram is non-canonical or evidence chain is incomplete.

Operational Fabric

Operational Fabric checks whether the structure may be used:

- host/scope binding;
- license admissibility;
- taint admissibility;
- secret and injection safety;
- non-copying/originality;
- host alignment;
- Foundry requirement;
- Workbench permitted use;
- memory promotion eligibility;
- human review requirement;
- phase stability / extrinsic commit readiness.

DF-OP-001
Operational Fabric must fail or downgrade if license, taint, injection, secret, originality or host-scope gates cannot be satisfied.

Chapter 17

Seam Checks, Mirror Consistency and AssimilationCertificate

Seam checks connect structural validity to operational admissibility.

```
pub struct SeamCheck {
    pub id: Digest,
    pub seam_kind: SeamKind,

    pub structural_ref: Digest,
    pub operational_ref: Digest,

    pub consistency_score: Q16,
    pub threshold: Q16,
    pub decision: SeamDecision,

    pub diagnostics: Vec<SeamDiagnostic>,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

```
pub enum SeamKind {
    HostScopeSeam,
    LicenseSeam,
    TaintSeam,
    OriginalitySeam,
    FoundrySeam,
    PhaseSeam,
    ReplaySeam,
    PromotionSeam,
}
```

MirrorConsistencyReport

```
pub struct MirrorConsistencyReport {
    pub id: Digest,

    pub structural_branch_digest: Digest,
    pub operational_branch_digest: Digest,

    pub seam_scores: Vec<Q16>,
```

```
    pub min_score: Q16,
    pub aggregate_score: Q16,

    pub required_threshold: Q16,
    pub no_float_audit_path: bool,

    pub decision: MirrorConsistencyDecision,
}
```

MIRROR-001
All audit-path numeric values affecting certificate digests, thresholds, seams and final decisions must use integer, rational or fixed-point encoding.

MIRROR-002
Mirror consistency failure must block StructuralCrystal commit.

AssimilationCertificate

```
pub struct AssimilationCertificate {
    pub id: Digest,
    pub certificate_version: String,

    pub run_id: Digest,
    pub null_center_digest: Digest,
    pub structural_crystal_candidate_id: Digest,

    pub structural_fabric_certificate: Digest,
    pub operational_fabric_certificate: Digest,
    pub seam_checks: Vec<SeamCheckRef>,
    pub mirror_consistency_report: Digest,

    pub replay_proof: ReplayProof,
    pub threshold_profile_id: Digest,

    pub final_decision: CertificationDecision,
    pub certified_permitted_targets: Vec<CertifiedTarget>,

    pub canonical_digest: Digest,
    pub evidence_refs: Vec<EvidenceRef>,
    pub created_at: Timestamp,
}
```

```
pub enum CertificationDecision {
    CertifiedForEvidenceOnly,
    CertifiedForCorpusCartography,
    CertifiedForStructuralCrystalArchive,
    CertifiedForNormGeneCandidate,
    CertifiedForPatternMemoryProposal,
    CertifiedForHumanReviewOnly,
    RejectCertification,
    Quarantine,
}
```

Chapter 18

ReplayProof and StructuralCrystalRecord

```
pub struct ReplayProof {
    pub id: Digest,

    pub input_digest: Digest,
    pub operator_stack_digest: Digest,
    pub policy_profile_digest: Digest,
    pub evidence_chain_digest: Digest,

    pub structural_branch_replay_digest: Digest,
    pub operational_branch_replay_digest: Digest,
    pub certificate_digest: Digest,

    pub replay_status: ReplayStatus,
}
```

CERT-REPLAY-001
AssimilationCertificate must include replay proof.

CERT-REPLAY-002
Operator versions affecting certificate outcome must be pinned.

CERT-REPLAY-003
ReplayIncompleteEvidence must block StructuralCrystalArchive commit unless explicitly routed as HumanReviewOnly.

Commit Predicate

```
EvidenceComplete
AND ReplayStable
AND StructuralValid
AND TopologyStable
AND PhaseAdmissible
AND DualFabricCertified
AND SeamConsistent
AND OperationallyPermitted
```

StructuralCrystalRecord

```
pub struct StructuralCrystalRecord {
    pub crystal_id: Digest,
    pub candidate_id: Digest,
    pub crystal_kind: StructuralCrystalKind,

    pub condensed_region: CodeHDAGFragment,
    pub constraint_program: ConstraintProgram,
    pub motif_core: MotifStructuralCore,

    pub stability_score: Q16,
    pub topology_signature: TopologySignature,
    pub cube_signature: CubeSignature,

    pub corpus_support: CorpusSupport,
    pub evidence_chain: Vec<EvidenceRef>,

    pub assimilation_certificate_id: Digest,
    pub operator_versions: BTreeMap<String, String>,

    pub created_at: Timestamp,
    pub lifecycle_state: CorpusEntityLifecycleState,
    pub supersedes: Option<Digest>,
}
```

CRYSTAL-IMM-001
Committed StructuralCrystalRecords are immutable.

CRYSTAL-IMM-002
Updating, replacing, weakening or strengthening a committed crystal must create a new record that supersedes the old one.

Chapter 19

Resonite and NormGene

A Resonite is the smallest neutral structural observation.

```
pub struct Resonite {
    pub id: Digest,
    pub kind: ResoniteKind,
    pub role: Option<RoleKind>,
    pub edge_kind: Option<CodeEdgeKind>,
    pub layer_kind: Option<CubeLayerKind>,
    pub fiber_kind: Option<FiberKind>,

    pub signature_shape: Option<SignatureShape>,
    pub attributes: BTreeMap<String, ObservationValue>,

    pub source_refs: Vec<EvidenceRef>,
    pub confidence: Q16,
}
```

```
pub enum ResoniteKind {
    Role,
    Edge,
    Layer,
    Fiber,
    Phase,
    Constraint,
    TestRelation,
    InterfaceBoundary,
    SecurityBoundary,
    ErrorRepairShape,
    DependencyPattern,
    AntiPatternSignal,
}
```

NormGene

```
pub struct NormGene {
    pub id: Digest,
    pub name: String,

    pub gene_kind: NormGeneKind,
    pub maturity: NormGeneMaturity,
```

```
    pub resonites: Vec<Resonite>,
    pub motif_core: MotifStructuralCore,
    pub constraint_program: ConstraintProgram,

    pub activation_conditions: Vec<ApplicabilityCondition>,
    pub compatibility: NormCompatibilityProfile,
    pub dependencies: Vec<NormGeneDependency>,
    pub conflicts: Vec<NormGeneConflict>,

    pub fitness: NormFitnessTrace,
    pub corpus_support: CorpusSupport,
    pub foundry_history: FoundryOutcomeHistory,
    pub workbench_use_history: WorkbenchUseHistory,

    pub failure_memory: Vec<NegativeEvidenceRef>,
    pub originating_crystals: Vec<StructuralCrystalRef>,
    pub evidence_refs: Vec<EvidenceRef>,

    pub lifecycle_state: NormGeneLifecycleState,
}
```

```
pub enum NormGeneKind {
    TestDesignGene,
    InterfaceBoundaryGene,
    LayeringGene,
    SecurityBoundaryGene,
    ErrorRepairGene,
    DependencyIntegrationGene,
    BuildPipelineGene,
    PluginArchitectureGene,
    AntiPatternGuardGene,
    BridgeGene,
}
```

NORMGENE-000
A NormGene is not a trusted norm.

Chapter 20

NormFitnessTrace and NormGene Lifecycle

```
pub enum NormGeneMaturity {
    Seed,
    Candidate,
    Recurrent,
    Certified,
    Mature,
    Deprecated,
    Rejected,
}
```

```
pub struct NormFitnessTrace {
    pub id: Digest,

    pub current_fitness: Q16,
    pub stability: Q16,
    pub host_fit_score: Q16,
    pub foundry_success_rate: Q16,
    pub workbench_reuse_rate: Q16,
    pub conflict_inverse: Q16,
    pub negative_evidence_penalty: Q16,
    pub freshness: Q16,

    pub events: Vec<NormFitnessEvent>,
}
```

```
pub enum NormFitnessEventKind {
    SupportedBySourceCube,
    SupportedByStructuralCrystal,
    UsedAsContextHint,
    FoundryPassed,
    FoundryFailed,
    HostAlignmentSucceeded,
    HostAlignmentFailed,
    ConflictDetected,
    SupersededByBetterGene,
    HumanApproved,
    HumanRejected,
    LicenseOrTaintDowngrade,
}
```

Fitness Update

```
fitness_next =
    alpha * fitness_current
  + (1 - alpha) * event_score
```

The formula is profile-configured and replayable. It is not an authority rule.

NORMFIT-001
Fitness updates must be replayable.

NORMFIT-002
Foundry failure must reduce fitness unless evidence shows the failure is unrelated to the NormGene.

NORMFIT-003
High fitness may increase retrieval priority but must not bypass gates.

Chapter 21

NormGene Birth, Activation and AntiPatternGenes

ActivationConditions

```
pub enum ApplicabilityConditionKind {
    HostVoidKind,
    DeficiencyKind,
    FrameworkOrEcosystem,
    LanguageProfile,
    LayerPresence,
    RolePresence,
    EdgePresence,
    MissingFiber,
    FoundryFailureClass,
    SecurityBoundaryRequirement,
    WorkbenchGoal,
}
```

Birth Conditions

```
pub struct NormGeneBirthCondition {
    pub min_structural_crystals: usize,
    pub min_independent_sources: usize,
    pub min_run_recurrence: usize,
    pub min_support_score: Q16,
    pub max_conflict_score: Q16,
    pub require_assimilation_certificate: bool,
    pub require_foundry_history: bool,
}
```

NORMGENE-BIRTH-001
A NormGeneCandidate must derive from at least one certified StructuralCrystal or from equivalent multi-run, multi-source, gate-passed evidence.

NORMGENE-BIRTH-002
Single-source support should not create a NormGeneCandidate.

NORMGENE-BIRTH-003
NormGene birth must record source diversity, run recurrence, host alignment, conflicts, Foundry requirement and certificate refs.

AntiPatternGene

```
pub struct AntiPatternGene {
    pub id: Digest,
    pub anti_pattern_kind: AntiPatternKind,
    pub observed_shape: MotifStructuralCore,
    pub avoidance_predicate: NormPredicateShape,
    pub risk_score: Q16,
    pub support: CorpusSupport,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

AntiPatternGenes are particularly important for future gates and collapse planning.

Chapter 22

HostTargetCollapsePlan

A HostTargetCollapsePlan turns HostTargetDelta into ordered structural action.

```
pub struct HostTargetCollapsePlan {
    pub id: Digest,
    pub run_id: Digest,

    pub host_cube_id: Digest,
    pub host_target_delta_id: Digest,
    pub corpus_cartography_ref: Option<Digest>,

    pub collapse_graph: CollapseGraph,
    pub steps: Vec<CollapseStep>,
    pub synchronized_groups: Vec<SynchronizedVoidGroup>,
    pub singularities: Vec<CollapseSingularity>,
    pub bisections: Vec<CollapseBisection>,

    pub expected_effect: CollapsePlanEffect,
    pub risk_profile: CollapsePlanRisk,
    pub validation_plan: CollapseValidationPlan,

    pub evidence_refs: Vec<EvidenceRef>,
}
```

COLLAPSE-000
HYPHAE v0.4 must not treat HostTargetDelta as an unordered improvement list.

Every non-trivial HostTargetDelta should be converted into a HostTargetCollapse-Plan that orders structural improvements by coupling, singularity impact, void coverage, risk, dependency and validation path.

CollapseDimension

```
pub enum CollapseDimensionKind {
    ArchitectureBoundary,
    LayerSeparation,
    InterfaceContract,
    TestFiber,
    SecurityBoundary,
    ErrorRepair,
```

```
    DependencyIntegration,
    BuildPipeline,
    DataModelBoundary,
    PluginBoundary,
    BridgeContract,
}
```

```
pub enum CollapseDimensionState {
    Free,
    FixedByHost,
    FixedByNormGene,
    FixedByStructuralCrystal,
    DerivedFromOtherDimension,
    RequiresHumanDecision,
    BlockedByGate,
}
```

Chapter 23

CollapseGraph, Couplings and Steps

```
pub struct CollapseGraph {
    pub id: Digest,
    pub dimensions: Vec<CollapseDimension>,
    pub couplings: Vec<CollapseCoupling>,
    pub spectral_profile: CollapseSpectralProfile,
    pub homology_profile: CollapseHomologyProfile,
}
```

```
pub enum CollapseCouplingKind {
    RequiresBefore,
    Enables,
    BlocksUntilResolved,
    CoChangesWith,
    TestsDependOn,
    SecurityDependsOn,
    InterfaceDependsOn,
    FoundryDependsOn,
    ConflictsWith,
}
```

CollapseStep

```
pub enum CollapseStep {
    Singularity {
        dimension: CollapseDimensionRef,
        options: Vec<CollapseOption>,
        impact: Q16,
        decision_requirement: DecisionRequirement,
    },

    Group {
        dimensions: Vec<CollapseDimensionRef>,
        synchronization_score: Q16,
        reason: SynchronizedGroupReason,
    },

    Bisect {
        left: Vec<CollapseDimensionRef>,
        right: Vec<CollapseDimensionRef>,
        fiedler_value: Q16,
```

```
        parallelizable: bool,
    },

    Leaf {
        dimension: CollapseDimensionRef,
        method: CollapseResolutionMethod,
    },

    Validate {
        target_steps: Vec<CollapseStepRef>,
        foundry_checks: Vec<FoundryCheck>,
    },
}
```

Resolution Methods

```
pub enum CollapseResolutionMethod {
    ExistingHostStructure,
    StructuralCrystal(Digest),
    NormGene(Digest),
    NormAssemblyCandidate(Digest),
    MotifGuidedWorkbenchPlan(Digest),
    FoundryRepairLoop(Digest),
    HumanDecisionRequired,
    EvidenceOnly,
}
```

COLLAPSE-PLAN-001
CollapsePlan is planning guidance, not direct code mutation authority.

Chapter 24

Singularities, Synchronized Groups and Validation

CollapseSingularity

```
pub struct CollapseSingularity {
    pub id: Digest,
    pub dimension: CollapseDimensionRef,
    pub affected_dimensions: Vec<CollapseDimensionRef>,
    pub impact_fraction: Q16,
    pub options: Vec<CollapseOption>,
    pub spectral_gap_signal: Q16,
    pub requires_operator_decision: bool,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

COLLAPSE-SING-001
CollapseSingularities should be resolved before low-coupling dimensions, because downstream plans may become invalid after singularity resolution.

SynchronizedVoidGroup

```
pub enum SynchronizationKind {
    SameLayerChain,
    SameTestFiber,
    SameSecurityBoundary,
    SameInterfaceContract,
    SameFoundryFailureClass,
    SameNormGeneActivation,
}
```

Validation

```
pub struct CollapseValidationCheckpoint {
    pub id: Digest,
    pub after_steps: Vec<CollapseStepRef>,
```

```
    pub required_checks: Vec<FoundryCheck>,
    pub expected_topology: Option<TopologySignature>,
    pub rollback_or_review_policy: ValidationFailurePolicy,
}
```

```
pub enum ValidationFailurePolicy {
    RetryWithMoreConstraints,
    DecomposeFurther,
    HumanReview,
    AbortPlan,
    PersistNegativeEvidence,
}
```

COLLAPSE-VALID-001
Every high-impact CollapseStep must declare validation checkpoints.

Chapter 25

NormAssembly

NormAssembly composes multiple NormGenes into cross-layer structure.

```
pub struct NormAssembly {
    pub id: Digest,
    pub name: String,

    pub level: NormAssemblyLevel,
    pub assembly_kind: NormAssemblyKind,

    pub genes: Vec<NormGeneRef>,
    pub layer_coverage: NormLayerCoverage,
    pub dependency_graph: NormAssemblyDependencyGraph,
    pub conflict_graph: NormAssemblyConflictGraph,

    pub activation: NormAssemblyActivation,
    pub compatibility: NormAssemblyCompatibility,

    pub expected_effect: AssemblyEffect,
    pub risk_profile: AssemblyRiskProfile,

    pub corpus_support: CorpusSupport,
    pub certificate_refs: Vec<AssimilationCertificateRef>,
    pub evidence_refs: Vec<EvidenceRef>,

    pub lifecycle_state: NormAssemblyLifecycleState,
}
```

```
pub enum NormAssemblyLevel {
    Atom,
    Molecule,
    Organism,
    Ecosystem,
}
```

```
pub enum NormAssemblyKind {
    TestArchitectureAssembly,
    InterfaceBoundaryAssembly,
    LayeredServiceAssembly,
    SecurityBoundaryAssembly,
    ErrorRepairAssembly,
    PluginArchitectureAssembly,
```

```
    BuildPipelineAssembly,
    ObservabilityAssembly,
    CrossServiceBridgeAssembly,
    AntiPatternGuardAssembly,
}
```

ASSEMBLY-001
NormAssembly must derive from NormGenes, StructuralCrystals or certified StructuralYields.

ASSEMBLY-002
NormAssembly must declare layer coverage, dependencies, conflicts, activation conditions, expected effect, risk and evidence.

ASSEMBLY-003
NormAssembly must not be treated as code generation authority.

Chapter 26

NormLayerCoverage and Assembly Planning

```
pub struct NormLayerCoverage {
    pub database: CoverageState,
    pub model: CoverageState,
    pub query: CoverageState,
    pub service: CoverageState,
    pub api: CoverageState,
    pub frontend: CoverageState,
    pub test: CoverageState,
    pub config: CoverageState,
    pub security: CoverageState,
    pub observability: CoverageState,
}
```

```
pub enum CoverageState {
    NotApplicable,
    Missing,
    Partial,
    CoveredByHost,
    CoveredByGene(NormGeneRef),
    CoveredByAssembly(NormAssemblyRef),
    RequiresHumanDecision,
}
```

AssemblyDependencyGraph

```
pub enum NormAssemblyDependencyKind {
    Requires,
    Enables,
    Configures,
    Validates,
    Tests,
    Guards,
    Bridges,
    Refines,
}
```

AssemblyConflictGraph

```
pub enum NormAssemblyConflictKind {
    LayerConflict,
    InterfaceConflict,
    SecurityConflict,
    DependencyConflict,
    RuntimeModelConflict,
    TestStrategyConflict,
    LicenseOrTaintConflict,
    FoundryHistoryConflict,
}
```

ASSEMBLY-CONFLICT-001
NormAssembly conflicts must be preserved and surfaced.

ASSEMBLY-CONFLICT-002
A high-conflict assembly must not become Workbench planning guidance without resolution, isolation or HumanReview.

Chapter 27

BridgeMotif, HostCluster and LandscapeTargetDelta

BridgeMotif is the cross-host connection motif of v0.4 Extended.

```
pub struct BridgeMotif {
    pub id: Digest,
    pub bridge_kind: BridgeKind,

    pub left_endpoint: BridgeEndpoint,
    pub right_endpoint: BridgeEndpoint,

    pub contract_shape: InterfaceContractMotif,
    pub communication_pattern: CommunicationPattern,
    pub security_boundary: Option<SecurityBoundaryMotif>,
    pub observability_pattern: Option<ObservabilityMotif>,

    pub required_genes: Vec<NormGeneRef>,
    pub required_assemblies: Vec<NormAssemblyRef>,

    pub compatibility: BridgeCompatibility,
    pub risk_profile: BridgeRiskProfile,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

```
pub enum BridgeKind {
    ApiGatewayBridge,
    EventBusBridge,
    SharedAuthBridge,
    SharedSchemaBridge,
    PluginBoundaryBridge,
    DataReplicationBridge,
    ObservabilityBridge,
    DeploymentBridge,
    MessageQueueBridge,
    CapabilityBoundaryBridge,
}
```

HostCluster

```
pub struct HostCluster {
    pub id: Digest,
    pub cluster_kind: HostClusterKind,

    pub host_cubes: Vec<Digest>,
    pub bridge_motifs: Vec<BridgeMotifRef>,
    pub cluster_hdag: ClusterHDAG,

    pub cluster_void_map: ClusterVoidMap,
    pub cluster_target_delta: Option<LandscapeTargetDelta>,

    pub evidence_refs: Vec<EvidenceRef>,
}
```

BRIDGE-001
BridgeMotif must connect at least two distinct HostCube regions or two distinct system regions.

BRIDGE-002
BridgeMotif must declare endpoint contracts, communication pattern, security or taint boundaries where applicable and validation requirements.

BRIDGE-003
BridgeMotif must not be promoted from single-host evidence alone.

Chapter 28

MorphogenicCorpusUpdate

MorphogenicCorpusUpdate governs how the long-term map changes.

```
pub struct MorphogenicCorpusUpdate {
    pub id: Digest,
    pub cartography_id: Digest,
    pub run_id: Digest,

    pub update_kind: MorphogenicUpdateKind,
    pub pressure_report: StructuralPressureReport,
    pub update_plan: MorphogenicUpdatePlan,

    pub applied_changes: Vec<MorphogenicChange>,
    pub replay_manifest: CartographyReplayManifest,
    pub invariant_checks: Vec<InvariantCheckResult>,

    pub decision: MorphogenicUpdateDecision,
    pub evidence_refs: Vec<EvidenceRef>,
}
```

```
pub enum MorphogenicUpdateKind {
    AddEntities,
    StrengthenRelations,
    WeakenRelations,
    MarkDormant,
    ReactivateDormant,
    SupersedeCrystal,
    SupersedeNormGene,
    PromoteCandidate,
    DemoteCandidate,
    ArchiveCold,
    CompactIndex,
    SplitCluster,
    FuseClusters,
    QuarantineRegion,
    MaskUnsafeRegion,
}
```

MORPH-000
HYPHAE v0.4 may adapt CorpusCartography, active indices, NormGene fitness,

relation weights, storage tiers and retrieval surfaces. It must not mutate committed truth.

StructuralPressureReport

```
pub struct StructuralPressureReport {
    pub recurring_void_pressure: Q16,
    pub repeated_failure_pressure: Q16,
    pub conflict_pressure: Q16,
    pub source_drift_pressure: Q16,
    pub license_taint_pressure: Q16,
    pub stale_relation_pressure: Q16,
    pub foundry_failure_pressure: Q16,
    pub storage_pressure: Q16,
    pub oracle_dependency_pressure: Q16,
    pub triggering_events: Vec<PressureEventRef>,
}
```

Chapter 29

Retention, Decay, Dormancy and Supersession

Retention Tiers

Hot Active runs, fresh SourceCubes, current Frontier traces, current MotifCandidates.

Warm Reusable motifs, active NormGenes, stable CorpusRelations, recent Foundry outcomes.

Cold StructuralCrystals, AssimilationCertificates, historical SourceCube signatures, superseded NormGenes, old ConflictRegions.

Quarantine Unsafe, taint-risk, license-blocked, injection-risk or secret-adjacent material.

RETENTION-001
StructuralCrystals, AssimilationCertificates and committed EvidenceChains should have indefinite retention unless policy explicitly requires archive or deletion handling.

Decay

```
pub enum DecayReason {
    StaleEvidence,
    FoundryFailure,
    ConflictIncrease,
    LowReuse,
    SourceUnavailable,
    LicenseDrift,
    FrameworkObsolete,
    BetterSupersedingCrystal,
}
```

DECAY-001
Decay must reduce active retrieval priority, not delete historical evidence.

Dormancy and Reactivation

Dormant entities remain queryable for audit and explicit retrieval. Default retrieval may exclude them.

Supersession

```
pub enum SupersessionKind {
    CrystalSupersedesCrystal,
    NormGeneSupersedesNormGene,
    NormAssemblySupersedesAssembly,
    AntiPatternGuardSupersedesGuard,
    BridgeMotifSupersedesBridge,
}
```

SUPERSESSION-001
Supersession must create a new record and link to the previous entity.

SUPERSESSION-002
The old entity remains replay-valid and historical.

Chapter 30

Masking, Quarantine, Compaction and Recovery

RetrievalMask

```
pub enum RetrievalMaskKind {
    HideFromDefaultRetrieval,
    HideFromWorkbenchContext,
    HideFromNormGeneBirth,
    HideFromCollapsePlanner,
    QuarantineOnly,
    HumanReviewOnly,
}
```

MASK-001
Masking must not erase evidence.

MASK-002
Quarantined material must not contribute to Workbench Context, NormGene birth, CollapsePlan or StructuralCrystal commit unless explicitly downgraded through safety review.

Compaction

```
pub enum CompactionKind {
    IndexOnlyCompaction,
    TimeSeriesDownsample,
    SourceSnapshotDigestOnly,
    SourceSnapshotArchiveExternalized,
    RelationRollup,
    SupersessionChainRollup,
    NegativeEvidenceSummary,
}
```

```
pub enum ReplayImpact {
    FullReplayPreserved,
    ReplayRequiresExternalArchive,
```

```
    ReplayDigestOnly,
    ReplayIncompleteDeclared,
}
```

COMPACT-001
Compaction must distinguish active-index compaction from evidence compaction.

COMPACT-002
Compaction must emit a CompactionRecord with before/after digests.

Storage Recovery

If hot or warm indices corrupt, HYPHAE should rederive them from cold StructuralCrystals, EvidenceBundles and ReplayManifests. If rederivation is impossible, affected regions must be quarantined or marked replay-incomplete.

Chapter 31

Implementation Architecture

v0.4 extends the v0.3 module map.

```
kosmo-hyphae/
  run.rs
  host.rs
  deficiency.rs
  frontier.rs
  acquisition.rs
  parsing.rs
  hdag.rs
  cube.rs
  cube_swarm.rs
  motif.rs
  structural_yield.rs
  gates.rs
  assimilation.rs
  integration.rs
  metrics.rs

  // v0.4 additions
  corpus_cartography.rs
  corpus_entity.rs
  corpus_relation.rs
  cartography_precheck.rs
  cartography_update.rs

  tritemporal_clock.rs
  phase_ladder.rs
  carrier_migration.rs

  structural_crystal.rs
  dual_fabric.rs
  assimilation_certificate.rs
  replay_proof.rs

  resonite.rs
  norm_gene.rs
  norm_fitness.rs
  norm_assembly.rs

  collapse_plan.rs
  collapse_graph.rs
```

```
  collapse_planner.rs

  morphogenic_update.rs
  retention.rs
  decay.rs
  supersession.rs
  quarantine.rs
  compaction.rs

  bridge_motif.rs
  host_cluster.rs
  landscape_delta.rs
```

Architecture Rule

ARCH-040
HYPHAE v0.4 extends v0.3 through a persistent morphogenic layer. It must not replace the v0.3 run-local CubeSwarm pipeline.

Chapter 32

Service Traits

CorpusCartographyStore

```
pub trait CorpusCartographyStore {
    fn load(&self, scope: &CorpusScope) -> Result<CorpusCartography,
   CartographyError>;

    fn query(&self, query: CartographyQuery)
        -> Result<CartographyQueryResult, CartographyError>;

    fn apply_update(&self, update: CorpusCartographyUpdate)
        -> Result<CartographyReplayManifest, CartographyError>;
}
```

TriTemporalRuntime

```
pub trait TriTemporalRuntime {
    fn initialize_clock(&self, input: ClockInitInput)
        -> Result<TriTemporalRunClock, TimeError>;

    fn record_intrinsic_event(&self, event: IntrinsicRunEvent)
        -> Result<(), TimeError>;

    fn commit_extrinsic_event(&self, event: ExtrinsicCommit)
        -> Result<ExtrinsicCommitRef, TimeError>;

    fn evaluate_migration(&self, report: PhaseHealthReport)
        -> Result<Option<CarrierMigration>, TimeError>;
}
```

CertificateEngine

```
pub trait CertificateEngine {
    fn build_candidate(
        &self,
        yield_item: &StructuralYield,
        context: &CertificationContext,
    ) -> Result<Option<StructuralCrystalCandidate>, CertError>;
```

```
    fn certify(
        &self,
        candidate: StructuralCrystalCandidate,
        context: &CertificationContext,
    ) -> Result<AssimilationCertificate, CertError>;
}
```

NormGeneEngine and CollapsePlanner

```
pub trait NormGeneEngine {
    fn evaluate_birth(
        &self,
        crystal: &StructuralCrystalRecord,
        cartography: &CorpusCartography,
    ) -> Result<Option<NormGeneCandidate>, NormGeneError>;

    fn update_fitness(
        &self,
        gene: &NormGene,
        event: NormFitnessEvent,
    ) -> Result<NormGene, NormGeneError>;
}
```

```
pub trait CollapsePlanner {
    fn build_collapse_graph(
        &self,
        input: CollapsePlanningInput,
    ) -> Result<CollapseGraph, CollapseError>;

    fn plan_collapse(
        &self,
        graph: &CollapseGraph,
        context: &CollapsePlanningContext,
    ) -> Result<HostTargetCollapsePlan, CollapseError>;
}
```

Chapter 33

Persistence Model

v0.4 uses three persistence levels:

1. **Working Store**: temporary, run-local, not trusted.
2. **Evidence / Ledger Store**: Kosmocrates-owned, append-only, replay-relevant.
3. **CorpusCartography Store**: HYPHAE-owned index surface, evidence-bound and non-authoritative.

PERSIST-040
CorpusCartography may store indices, relations, maturity and retrieval state. It must not store unproven trusted norms.

PERSIST-041
All durable CorpusCartography updates must reference Kosmocrates Evidence-Bundles and replay manifests.

PERSIST-042
StructuralCrystals and AssimilationCertificates are immutable after commit.

Storage Layout

```
hyphae/
  working/
    runs/<run_id>/
    sandboxes/
    transient_swarms/

  evidence/
    bundles/
    ledger_events/
    replay_manifests/

  cartography/
    scopes/<scope_id>/
      source_cube_index/
      motif_index/
      norm_gene_index/
      negative_evidence_index/
```

```
    structural_crystals/
    morphogenic_updates/
    quarantine/
```

Chapter 34

API Contract

Existing v0.3 endpoints remain valid. v0.4 adds:

```
GET  /hyphae/cartography/{scope}
GET  /hyphae/cartography/{scope}/motifs
GET  /hyphae/cartography/{scope}/norm-genes
GET  /hyphae/cartography/{scope}/crystals
GET  /hyphae/cartography/{scope}/negative-evidence

GET  /hyphae/runs/{run_id}/clock
GET  /hyphae/runs/{run_id}/phase-ladder
GET  /hyphae/runs/{run_id}/collapse-plan
GET  /hyphae/runs/{run_id}/certificates

POST /hyphae/cartography/{scope}/morphogenic-update
POST /hyphae/norm-genes/{id}/review
POST /hyphae/crystals/{id}/supersede
```

MVP API Minimal

```
GET /hyphae/runs/{run_id}/collapse-plan
GET /hyphae/runs/{run_id}/certificates
GET /hyphae/cartography/{scope}/negative-evidence
GET /hyphae/cartography/{scope}/norm-genes
```

CLI

```
kosmo hyphae cartography inspect --scope local
kosmo hyphae cartography negative --scope local
kosmo hyphae cartography norm-genes --scope local

kosmo hyphae run --host . --goal fill-missing-tests --v04
kosmo hyphae collapse-plan <run-id>
kosmo hyphae certificates <run-id>
kosmo hyphae norm-gene show <gene-id>
kosmo hyphae morph update --scope local
```

Chapter 35

v0.4 MVP Cut

The v0.4 MVP must prove persistent, certified learning over the v0.3 CubeSwarm pipeline.

MUST Include

- CorpusCartographyStore;
- CartographyPrecheck;
- SourceCubeIndex;
- MotifIndex;
- NegativeEvidenceIndex;
- minimal StructuralCrystalArchive;
- minimal TriTemporalRunClock;
- PhaseLadder with Primary, TestsOnly and PriorEvidenceOnly;
- FrictionReport / ShockReport;
- StructuralCrystalCandidate;
- minimal DualFabricGateCascade;
- AssimilationCertificate;
- Resonite;
- NormGeneCandidate;
- NormFitnessTrace;
- HostTargetCollapsePlan;
- minimal MorphogenicCorpusUpdate;
- ReplayManifest.

May Defer

- full BridgeMotif runtime;
- HostCluster assimilation;
- LandscapeTargetDelta;
- full NormAssembly planner;
- spectral Fiedler implementation;
- Kuramoto synchronization;

- persistent homology implementation;
- complex carrier splitting;
- automatic crystal supersession;
- Studio visualizations.

MUST NOT

- direct trusted norm promotion;
- autonomous host mutation;
- raw external code import;
- CorpusCartography bypassing gates;
- NormGene bypassing Foundry;
- certificate without replay proof;
- morphogenic update without before/after digest.

Chapter 36

v0.4 MVP Scenario

Scenario

A Rust web API host repository has missing integration tests and weak layer boundaries.

Expected Flow

1. HostCube and VoidMap detect MissingTestFiber and WeakServiceLayer.
2. CartographyPrecheck searches existing TestDesignGenes.
3. If insufficient, v0.3 SourceFrontier and CubeSwarm run.
4. StructuralYield emits TestDesignMotif and LayeringMotif.
5. DualFabricGateCascade certifies eligible CrystalCandidates.
6. CorpusCartography stores Crystal, Motif and NegativeEvidence.
7. NormGeneCandidate is born for RouteIntegrationTestGene.
8. HostTargetCollapsePlan orders RepositoryPort/ServiceBoundary first, IntegrationFixture second, NegativeCases third.
9. Workbench receives a PlanningArtifact.
10. FoundryValidationRequest is created.
11. MorphogenicCorpusUpdate writes before/after replay manifest.

Success Criteria

- No raw external code enters ContextPack.
- CollapsePlan is typed and validatable.
- AssimilationCertificate contains ReplayProof.
- NegativeEvidence is retrieval-visible.
- NormGeneCandidate is not a trusted norm.
- CorpusCartography is append-only updated.

Chapter 37

Implementation Phases

Phase	Name	Definition of Done
0	v0.4 Spec Freeze	v0.4 Master Spec, Type Catalog, Gate/CERT Catalog, MVP Cut and migration notes are accepted.
A	CorpusCartography Minimal	A run can persist positive and negative results; a later run can use them in Precheck.
B	TriTemporal Runtime Minimal	A run can migrate from bad Primary to TestsOnly or PriorEvidenceOnly and preserve abandoned phase evidence.
C	Certification	StructuralYield can be certified, rejected or downgraded with replayable certificate.
D	NormGene Minimal	Certified TestDesignCrystal can seed NormGeneCandidate, but not trusted Norm.
E	CollapsePlan	HostTargetDelta becomes ordered Workbench task sequence with Foundry checkpoints.
F	Morphogenic Governance	CorpusCartography can grow, decay, dormancy-mark and supersede without mutating committed history.
G	API / CLI / Reports	Operator can inspect Precheck, PhaseLadder, Certificates, CollapsePlan, NormGeneCandidate and MorphogenicUpdate.
H	NormAssembly Extended	NormGenes can be composed into cross-layer assemblies.
I	BridgeMotif Extended	HostCluster and BridgeMotifs support multi-host planning.

Chapter 38

v0.4 Metrics

```
pub struct HyphaeV04Metrics {
    pub cartography: CartographyMetrics,
    pub tritemporal: TriTemporalMetrics,
    pub certification: CertificationMetrics,
    pub norm_gene: NormGeneMetrics,
    pub collapse: CollapsePlanMetrics,
    pub morphogenic: MorphogenicMetrics,
}
```

CartographyMetrics

```
pub struct CartographyMetrics {
    pub prior_motif_hit_rate: Q16,
    pub negative_evidence_reuse_rate: Q16,
    pub search_reduction_rate: Q16,
    pub norm_gene_hit_rate: Q16,
}
```

CertificationMetrics

```
pub struct CertificationMetrics {
    pub crystal_candidates_total: usize,
    pub certificates_issued: usize,
    pub certification_rejection_rate: Q16,
    pub seam_failure_rate: Q16,
    pub replay_failure_rate: Q16,
}
```

CollapsePlanMetrics

```
pub struct CollapsePlanMetrics {
    pub collapse_dimensions: usize,
    pub singularity_count: usize,
    pub synchronized_group_count: usize,
    pub expected_void_reduction: Q16,
    pub expected_oracle_reduction: Q16,
```

```
}
```

MorphogenicMetrics

```
pub struct MorphogenicMetrics {
    pub updates_applied: usize,
    pub entities_decayed: usize,
    pub entities_reactivated: usize,
    pub supersessions: usize,
    pub replay_incomplete_count: usize,
}
```

Success Criteria

v0.4 succeeds if later runs safely reuse prior evidence, negative evidence reduces waste, StructuralYields can be certified or rejected replayably, NormGeneCandidates mature without becoming trusted norms, CollapsePlans improve Workbench planning, morphogenic updates preserve history, Foundry feedback changes fitness and no corpus signal bypasses gates.

Chapter 39

Acceptance Tests

1. **V04-A1:** CartographyPrecheck finds prior NegativeEvidence and reduces Frontier ranking.
2. **V04-A2:** A second run against similar MissingTestFiber uses prior NormGeneCandidate as retrieval hint but still runs gates.
3. **V04-A3:** Low-yield Primary phase migrates to TestsOnly phase and preserves abandoned phase evidence.
4. **V04-A4:** StructuralYield without complete evidence cannot become StructuralCrystalCandidate.
5. **V04-A5:** DualFabric certification fails when Structural Fabric passes but Operational Fabric fails due to license or taint.
6. **V04-A6:** AssimilationCertificate contains ReplayProof, branch digests and seam checks.
7. **V04-A7:** Certified StructuralCrystal can seed NormGeneCandidate.
8. **V04-A8:** Foundry failure lowers NormGene fitness and creates failure memory.
9. **V04-A9:** HostTargetCollapsePlan orders high-coupling layer/interface steps before test-only leaves.
10. **V04-A10:** MorphogenicUpdate supersedes old Crystal without mutating old Crystal.
11. **V04-A11:** Dormant entity is excluded from default retrieval but remains audit-queryable.
12. **V04-A12:** Compaction emits replay impact and before/after digest.
13. **V04-A13:** CorpusCartography hit never bypasses GateCascade.
14. **V04-A14:** NormGene maturity never equals trusted Norm promotion.
15. **V04-A15:** Full v0.4 MVP run emits EvidenceBundle, Certificate, CollapsePlan, NormGeneCandidate, CorpusCartographyUpdate and RunReport.

Chapter 40

v0.4 System Definition of Done

DOD-040 v0.4 preserves all v0.3 DoD requirements.

DOD-041 Every durable cartography update has before/after digest and replay manifest.

DOD-042 Every StructuralCrystalRecord has AssimilationCertificate.

DOD-043 Every AssimilationCertificate has structural fabric, operational fabric, seam checks and replay proof.

DOD-044 NormGene is never treated as trusted norm by existence.

DOD-045 CollapsePlan is planning guidance, not mutation authority.

DOD-046 Morphogenic updates cannot mutate committed crystals, certificates or evidence.

DOD-047 CorpusCartography hits cannot bypass GateCascade.

DOD-048 Foundry-relevant durable structures carry validation requirements.

DOD-049 Negative evidence and failed certification outcomes are retrieval-visible.

DOD-050 A full MVP run emits EvidenceBundle, CorpusCartographyUpdate, AssimilationCertificate, NormGeneCandidate, HostTargetCollapsePlan, MorphogenicReplayRecord and RunReport.

Chapter 41

Requirement Catalog

ID	Requirement
NEUT-001	Source architectures contribute invariants, not original systems.
DOCTRINE-040	HYPHAE v0.4 remembers structure, not authority.
DOCTRINE-041	CorpusCartography cannot bypass gates or governance.
COMPAT-040	v0.4 extends the v0.3 pipeline and must not replace it.
CARTO-000	CorpusCartography is append-only, evidence-bound and non-trusted.
CARTO-REPLAY-001	Same prior digest and inputs should produce same after digest.
TIME-EXT-001	Only ExtrinsicCommit events may mutate durable surfaces.
MIG-002	CarrierMigration must not downgrade safety policy.
CRYSTAL-001	Discovery does not create a StructuralCrystal.
DF-STRUCT-001	Structural Fabric fails on invalid HDAG/core/constraints/evidence.
DF-OP-001	Operational Fabric fails/downgrades on unresolved safety/scope issues.
MIRROR-001	Audit path numeric values must use fixed-point/integer/rational encoding.
CERT-REPLAY-001	AssimilationCertificate must include ReplayProof.
NORMGENE-000	A NormGene is not a trusted norm.
NORMFIT-003	High fitness may increase priority but not bypass gates.
COLLAPSE-000	HostTargetDelta should become ordered CollapsePlan.
COLLAPSE-PLAN-001	CollapsePlan is not mutation authority.
ASSEMBLY-003	NormAssembly is not code generation authority.
BRIDGE-003	BridgeMotif must not be promoted from single-host evidence alone.
MORPH-000	Morphogenic updates may alter active views, not committed truth.
SUPERSESSION-001	Supersession creates new records and links to previous entities.
DOD-050	Full MVP run emits all v0.4 core outputs.

Chapter 42

Gate and Certificate Catalog

v0.3 GateCascade Preserved

```
G0  Run / Genesis Gate
G1  Host Binding Gate
G2  Source Evidence Gate
G3  Provenance Completeness Gate
G4  License Gate
G5  Secret Gate
G6  Injection Gate
G7  Taint Gate
G8  Non-Copying / Originality Gate
G9  Topology Validity Gate
G10 Cube Resonance Gate
G11 Host Alignment Gate
G12 Conflict Gate
G13 Novelty / Duplication Gate
G14 Workbench Permitted-Use Gate
G15 Memory Promotion Eligibility Gate
G16 Foundry Requirement Gate
G17 Human Review Gate
G18 Final Assimilation Decision
```

v0.4 Structural Fabric Gates

```
S0 Evidence chain complete
S1 CodeHDAG fragment valid
S2 MotifStructuralCore typed
S3 CubeSignature valid
S4 ConstraintProgram canonical
S5 Corpus support sufficient
S6 Conflict below threshold
S7 Replay digest stable
S8 Operator versions pinned
```

v0.4 Operational Fabric Gates

```
O0 Host / scope binding
O1 License admissibility
O2 Taint admissibility
O3 Secret and injection safety
O4 Non-copying / originality
O5 Host alignment
O6 Foundry requirement
O7 Workbench permitted use
O8 Memory promotion eligibility
O9 Human review requirement
O10 Phase stability / extrinsic commit readiness
```

Certificate Rigor Levels

```
R0 EvidenceOnly
R1 RunLocalReusable
R2 CorpusIndexable
R3 CrystalCandidate
R4 StructuralCrystalCommitted
R5 NormGeneCandidate
R6 NormPromotionEligible
```

Chapter 43

v0.3 to v0.4 Delta Table

Area	v0.3	v0.4
Learning	Positive/negative outcomes retrieval-visible	Persistent CorpusCartography
Time Model	Run pipeline	TriTemporalRunClock
Worker Robustness	Deterministic CubeSwarm	PhaseLadder + CarrierMigration
Yield Hardening	GateCascade	DualFabricGateCascade + AssimilationCertificate
Long-term Structure	NormCandidateDraft	StructuralCrystal + NormGene
Planning	HostTargetDelta	HostTargetCollapsePlan
Composition	Individual Motifs	NormAssembly
Multi-Host	Not central	BridgeMotif / HostCluster Extended
Evolution	Metrics / LearningTrace	MorphogenicCorpusUpdate
Truth	EvidenceBundle / GateTrace	ReplayManifest + immutable Crystals + Supersession

Closing Thesis

HYPHAE v0.3 extracts structural advantage from many repositories for one host run.

HYPHAE v0.4 makes that advantage cumulative.

It remembers what worked, what failed, what matured, what decayed, what must be superseded, what can be certified, what can become a NormGene, and in which order the host should collapse its remaining structural voids.

It remains non-copying, evidence-first, replay-bound, Foundry-bound and Kosmocrates-governed.

> v0.3 = topological condenser.
> v0.4 = morphogenic structural memory with certification and collapse planning.

Appendix A

PlantUML Diagrams

v0.4 Run Automaton

```
@startuml
start
:Load RunDescriptor;
:Initialize TriTemporalRunClock;
:Bind HostCube;
:Compute VoidMap + DeficiencyVector;
:CartographyPrecheck;
if (prior evidence sufficient?) then (yes)
  :Reuse certified motifs / NormGenes;
else (no)
  :Build SourceFrontier;
  :Acquire / parse / SourceCube;
  :CubeSwarm + Mandorla + Coagulation;
endif
:MotifExtraction -> StructuralYield;
:Run v0.3 GateCascade;
if (durable value?) then (yes)
  :Build StructuralCrystalCandidate;
  :DualFabricGateCascade;
  :Emit AssimilationCertificate;
  if (certified?) then (yes)
    :Append StructuralCrystalRecord;
    :Update CorpusCartography;
    :Evaluate NormGene birth/update;
  else (no)
    :Route to Evidence / Negative / Review;
  endif
endif
:Build HostTargetCollapsePlan;
:Route Workbench / Foundry / Ledger outputs;
:MorphogenicCorpusUpdate;
:RunReport + Metrics;
stop
@enduml
```

Structural Certification

```
@startuml
start
:Load StructuralYield;
if (long-lived assimilation needed?) then (yes)
  :Build StructuralCrystalCandidate;
  :Build ConstraintProgram;
  fork
    :Run Structural Fabric;
  fork again
    :Run Operational Fabric;
  end fork
  :Compute SeamChecks;
  :Compute MirrorConsistencyReport;
  :Build ReplayProof;
  if (Commit predicate satisfied?) then (yes)
    :Emit AssimilationCertificate;
    :Append StructuralCrystalRecord;
  else (no)
    :Emit rejection certificate;
    :Route to Evidence/Negative/Review;
  endif
else (no)
  :Use v0.3 CandidateEvidence path;
endif
stop
@enduml
```

HostTargetCollapsePlan

```
@startuml
start
:Load HostTargetDelta;
:Create CollapseDimensions;
:Build CollapseGraph;
:Detect Singularities;
:Compute SynchronizedVoidGroups;
:Compute Bisections;
:Identify NormGene / Crystal leaves;
:Order CollapseSteps;
:Add Foundry checkpoints;
:Emit WorkbenchPlanningArtifact;
stop
@enduml
```

Appendix B

MVP Run Output Checklist

A complete v0.4 MVP run should produce:

- EvidenceBundle;
- TriTemporalRunClock;
- CartographyPrecheck;
- SourceFrontierGraph or ExistingEvidenceSufficient decision;
- SourceEvidence and SourceCubes if acquisition ran;
- StructuralYield;
- GateCascade;
- StructuralCrystalCandidate where durable value exists;
- AssimilationCertificate;
- CorpusCartographyUpdate;
- NormGeneCandidate if birth conditions pass;
- HostTargetCollapsePlan;
- WorkbenchPlanningArtifact;
- FoundryValidationRequest where executable influence exists;
- MorphogenicReplayRecord;
- RunReport with v0.4 metrics.

Appendix C

Non-Goals

The following are explicitly out of scope for v0.4 MVP:

- unbounded crawling;
- autonomous host mutation;
- raw external code import;
- direct trusted norm promotion;
- CorpusCartography as authority;
- full multi-host landscape planning;
- automatic BridgeMotif promotion;
- automatic crystal supersession without governance;
- Studio visualization;
- mathematical completeness of spectral/homology planning.

Appendix D

Final Implementation Maxim

Build v0.3 first as the deterministic run-local assimilation engine. Then build v0.4 as the evidence-bound persistent layer that remembers, certifies, matures, plans and governs the structural knowledge discovered by v0.3.